



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The State Space refers to the complete set of positions or configurations the agent could ever occupy in this grid game, that means every legal coordinate the agent is allowed to move into.

The Search Tree, on the other hand, doesn't exist until the algorithm actually starts running it's built dynamically as the search explores outward from the start position, branching toward possible paths to the goal.

Put simply: the state space is the map of everything that *could* be visited, while the search tree is the record of what the algorithm *actually* explores while hunting for a solution.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set records which states the agent has already explored, so the algorithm doesn't waste effort revisiting the same coordinates repeatedly.

Without it, DFS is especially vulnerable it could cycle back and forth between previously visited cells indefinitely, essentially spinning in a loop. That burns through time and memory pointlessly, and in the worst case the agent never reaches the food at all.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands outward uniformly, examining all nodes at one distance before moving further out. Since every move in this grid costs the same amount, that level-by-level approach guarantees the first path BFS finds to the food is the shortest one.

DFS behaves very differently it commits to a single path and pushes as deep as possible before backtracking. That means it can stumble onto the goal via a long, roundabout route even when a much shorter path was sitting right there.

This matched what was observed in the simulation: BFS consistently picked efficient, direct paths and cleared all the food, while DFS wandered along longer routes and ran out of the 60-step budget before finishing.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS is memory-hungry because it has to hold onto every node at the current frontier level before it can advance its space complexity scales as $O(b^d)$, with b being the branching factor and d the depth of the nearest solution. That growth becomes unmanageable fast on something like a 1000×1000 grid.

DFS is much lighter on memory since it only needs to track the current path plus a handful of unexplored branches, giving it a space complexity closer to $O(bm)$, where m is the deepest level reached.

The tradeoff: on a huge grid, BFS is the one more likely to exhaust memory and crash, whereas DFS holds up better memory-wise but at the cost of no longer guaranteeing it'll find the shortest path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

